

INTERVIEW PREPARATION

React Hooks, Forms & TypeScript

A complete study guide built from the `form-practice` project — controlled components, state typing, Context, and the re-render model.

useState

useContext

useMemo

useCallback

Custom Hooks

Controlled Inputs

TypeScript

Next.js App Router

Project: form-practice · Next.js 16.2 · React 19.2 · TypeScript 5

Covers: every concept from building the Storing Tasks form end to end

Contents

1 Hooks — The Foundation

What a hook is, why the name, the rules, and how useState really works

2 Controlled Components & Forms

value, checked, onChange, labels, and native form submission

3 TypeScript for React Forms

Typing state, literal unions, event types, and decoding error messages

4 JavaScript Essentials That Bite

Object literals, identity, shallow spread, arrow-function returns, what returns objects vs arrays

5 The Context API

Prop drilling, the three pieces, the null guard, App Router rules

6 Re-renders & Optimisation

What triggers renders, how memoisation works internally, seven worked scenarios, stale closures, the checklist

7 Next.js App Router Notes

Server vs Client Components, 'use client', private folders

8 Rapid-Fire Interview Q&A

Short, memorisable answers to the most likely questions

9 Quick Reference

Cheat-sheet tables for last-minute revision

1 Hooks — The Foundation

1.1 What is a hook?

Your component is a plain JavaScript function that returns JSX. **React calls it again on every render** — every keystroke, every click. Plain variables therefore reset each time:

```
const Home = () => {  
  let name = '' // runs again on EVERY render → always resets  
  return <input value={name} onChange={(e) => { name = e.target.value }} />  
}
```

A component needs values that *survive* across calls, but can't use module scope (every instance would share one value). Hooks solve exactly this.

Interview answer A hook is a function that lets a component **"hook into" React's internal machinery** — state, lifecycle, context — from a plain function component. That's where the name comes from: your function has no access to React's internals on its own, and a hook is the attachment point.

Why they exist historically

Before React 16.8 (2019), state required **class components** (`this.state` , `componentDidMount`). Hooks brought all of it to plain functions — which is why older tutorials look completely different.

The hook family

Hook	Hooks into...
<code>useState</code>	memory that survives re-renders
<code>useEffect</code>	the moment after render — fetching, timers, subscriptions
<code>useContext</code>	a value broadcast by an ancestor component
<code>useRef</code>	a value that persists but does <i>not</i> trigger re-renders
<code>useMemo</code>	a cached computed value
<code>useCallback</code>	a cached function identity

All hooks start with `use` . That's a hard convention — the ESLint rules depend on it, and it's how you name your own **custom hooks**.

1.2 The Rules of Hooks

1. **Only call hooks at the top level** — never inside `if` , loops, or nested functions.
2. **Only call them from components or custom hooks** — not ordinary functions.

Why rule 1 exists — a favourite interview follow-up

React identifies your state by call order, not by variable name. It doesn't know the variable is called `formValues` — it knows "the first `useState` in this component" and "the second one".

```
const [formValues, setFormValues] = useState(defaultValues) // slot 1
const [taskList, setTaskList] = useState([]) // slot 2
```

If a condition made slot 1 disappear on some render, `taskList` would slide into position 1 and receive the wrong data. Hence: **same hooks, same order, every render.**

1.3 `useState` in depth

```
const [formValues, setFormValues] = useState<FormValues>(defaultValues);
```

Read it as three things:

1. `useState(defaultValues)` — "React, reserve a memory slot. **First time only**, put this in it."
2. `formValues` — what's in the slot right now.
3. `setFormValues` — the only legal way to change it.

The initial value is used **once**. On render #2, #3, #50, React ignores it entirely and returns the stored value.

Destructuring gotcha

```
const {formValues, setFormValues} = useState() // X object destructuring
const [formValues, setFormValues] = useState() // ✓ useState returns an ARRAY
```

`useState` returns a two-element array (a tuple): `[value, setter]`. Using `{}` looks for properties with those literal names — both `undefined`.

1.4 Why a setter instead of assignment

```
formValues.name = "Ali" // X nothing happens on screen
setFormValues(prev => ({...prev, name: "Ali"})) // ✓ screen updates
```

The setter does **two** jobs:

1. Store the new value
2. **Tell React to re-render this component**

Direct assignment only does job 1. React never finds out and never re-renders. **The notification is the whole point of the setter.**

1.5 The updater function — `prev =>`

Both of these look equivalent, but only one is always correct:

```
setCount(count + 1)           // reads a SNAPSHOT of state
setCount(prev => prev + 1)     // React hands you the LATEST value
```

`count` is a snapshot — a photo taken when the component rendered. It does not update mid-handler:

```
// count is 0
setCount(count + 1) // 0 + 1 = 1
setCount(count + 1) // count is STILL 0 here → 0 + 1 = 1
// final: 1 ✗ expected 2

// with the updater:
setCount(prev => prev + 1) // React passes 0 → returns 1
setCount(prev => prev + 1) // React passes 1 → returns 2 ✓
```

Remember this — it pays off twice The updater form matters again in **Part 6**: because `prev =>` reads nothing from the current render, it's what makes an empty `useCallback` dependency array safe. Two lessons, one idea.

1.6 Custom hooks

A custom hook is just a **function starting with `use` that calls other hooks**. No special API — the naming convention is what makes the linter treat it as a hook.

```
export const useTaskContext = () => {
  const context = useContext(TaskContext);
  if (!context) {
    throw new Error("useTaskContext must be used within a TaskProvider");
  }
  return context;
}
```

Why bother? It packages **logic + safety checks** so every consumer gets them for free — here, a clear error message and a type that no longer includes `null`.

2 Controlled Components & Forms

2.1 Controlled vs uncontrolled

Every input has built-in memory. The only question is **who owns the value**.

	Uncontrolled	Controlled
Owner	the DOM / browser	React state
Markup	<code><input /></code> or <code>defaultValue</code>	<code>value</code> + <code>onChange</code>
Reading it	ask the DOM via a <code>ref</code>	it's already in state
Best for	simple forms, performance, file inputs	validation, live UI, resets

Interview answer A controlled component is an input whose value lives in **React state** instead of the DOM. The input *displays* state, events *request* changes to state, and updated state *re-paints* the input. You're demoting the input from a container to a display screen.

The loop — one-way, not two-way binding

```
formValues.name = "Rosha" ↓ value={...} input displays "Rosha" ↓ user presses "n" onChange fires with  
"Roshan" ↓ setFormValues(...) formValues.name = "Roshan" ↓ re-render input displays "Roshan"
```

The letter does not appear because you typed it. The keystroke is intercepted, routed through state, and the input updates only because React re-rendered it with a new `value`. Every character is a round trip.

Proof — try it

```
<input type="checkbox" checked={false} onChange={() => {}} />
```

Click it. **Nothing happens.** The browser ticks it, your handler does nothing, React re-renders and forces `checked={false}` back. The browser's change is only a *suggestion* — state is law. This is why React warns "You provided a `checked` prop without an `onChange` handler."

2.2 Which prop binds which input

Input	Controlled	Uncontrolled	Read in handler
text / number	<code>value</code> + <code>onChange</code>	<code>defaultValue</code>	<code>e.target.value</code>
textarea	<code>value</code> + <code>onChange</code>	<code>defaultValue</code>	<code>e.target.value</code>
select	<code>value</code> on the <code><select></code>	<code>defaultValue</code>	<code>e.target.value</code>
checkbox	<code>checked</code> + <code>onChange</code>	<code>defaultChecked</code>	<code>e.target.checked</code>
radio	<code>checked</code> + <code>onChange</code>	<code>defaultChecked</code>	<code>e.target.checked</code>

select: value goes on the parent In plain HTML you'd write `<option selected>`. React deliberately doesn't work that way — one value on the `<select>` controls the whole group, exactly like one `name="gender"` controls three radios.

2.3 Why radios need `===` but checkboxes don't

checked always needs a boolean. The only question is whether your state already holds one.

State shape for that input	Binding
<code>boolean</code> per box	<code>checked={formValues.addons.overtime}</code> — pass it straight
one shared <code>string</code>	<code>checked={formValues.gender === "male"}</code> — convert
<code>string[]</code>	<code>checked={addons.includes("overtime")}</code> — membership test

Why the shapes differ: checkboxes are *independent* (overtime says nothing about lunch) → separate booleans. Radios are *mutually exclusive* (exactly one, always) → one variable holding the winner's name.

The deeper point Three booleans for gender would let you reach `{male: true, female: true}` — a state that's impossible in reality. One string makes invalid states **unrepresentable**. And `checked` is *derived*, never stored: one string decides all three via `===`, so they can't drift out of sync.

2.4 Labels, ids and names — three different jobs

```
<label htmlFor="name">Name</label>
<input id="name" name="name" ... />
```

Attribute	Job
<code>htmlFor</code>	links a <code><label></code> to an input — JSX name for HTML's <code>for</code> (<code>for</code> is a reserved JS keyword)
<code>id</code>	the target of <code>htmlFor</code> ; must be unique on the page
<code>name</code>	the key used on form submission ; also groups radios together

Linking a label means clicking the text focuses/toggles the input, and screen readers announce it. **An input with no `name` is not submitted at all** — a very common silent bug.

2.5 Native form submission — and why the page reloads

A `<form>` with no `onSubmit` does what forms have done since 1993: *collect the fields, put them in a URL, and navigate to it*.

- **No `action`** → submit to the current URL
- **No `method`** → defaults to `GET` → data goes in the query string

```
// result of clicking submit on an unhandled form:
http://localhost:3000/?gender=male&overtime=on
```

Navigation means a full page load, which wipes all React state.

Markup	Submitted as
input with <code>id</code> but no <code>name</code>	nothing — invisible to the form
checkbox, ticked, no <code>value</code>	<code>overtime=on</code> — <code>"on"</code> is the HTML default
checkbox, ticked, <code>value="overtime"</code>	<code>overtime=overtime</code>
checkbox, unticked	absent entirely — never sends <code>false</code>
3 radios sharing <code>name="gender"</code>	only the selected one

The fix

```
const handleSubmit = (e: React.SubmitEvent<HTMLFormElement>) => {
  e.preventDefault();      // cancels the navigation + reload
  addTask(formValues);      // use YOUR state — properly typed
  setFormValues(defaultValues);
}
```

Put `onSubmit` on the `<form>`, not the button — then pressing Enter in a text field works too, which users expect.

2.6 The five places every field lives

Adding one field (e.g. `role`) touches five spots. Memorise this checklist:

#	Where	What
1	the type	<code>type Role = "admin" "user"</code>
2	the interface	<code>role: Role</code>
3	the defaults	<code>role: 'user'</code>
4	the UI	a controlled input bound to it
5	the display	rendering it in the saved list

Why typing the defaults pays off Because `defaultValues` is annotated : `FormValues`, adding a field to the interface **immediately breaks the defaults** until you fill it in. The compiler walks you through the checklist.

3 TypeScript for React Forms

3.1 Three ways to type useState

```
// 1. Inference - TS reads the initial value. Best for primitives.
const [name, setName] = useState('') // string

// 2. Explicit generic - REQUIRED when the initial value can't describe the type
const [form, setForm] = useState<FormValues>(defaults)
const [user, setUser] = useState<User | null>(null) // else infers `null` forever
const [tasks, setTasks] = useState<FormValues[]>([]) // else infers `never[]`

// 3. Bare - infers `undefined`. Almost never useful.
const [x, setX] = useState()
```

Rule of thumb If the initial value is `null`, `undefined`, or `[]`, you **must** pass the generic — TypeScript has nothing to infer from.

3.2 Default values

```
const defaultValues: FormValues = {
  name: '', // '' not undefined
  task: '',
  role: 'user', // a union has no empty member
  gender: 'none',
  addons: { overtime: false, lunch: false, coffee: false },
}
```

- An empty form isn't "nothing" — it's an object whose fields are empty. `''` and `false` are the real starting values.
- `value={undefined}` starts **uncontrolled** then flips to controlled → the "changing an uncontrolled input to be controlled" warning.
- A named constant gives you a free reset: `setFormValues(defaultValues)`.

3.3 Literal types and unions

```
type Gender = "male" | "female" | "none";
```

Normally `"male"` is a *value*. Here it's used as a **type** — one so specific it has exactly one possible value.

```
let a: string = "male" // a can be any text at all
let b: "male" = "male" // b can ONLY ever be "male"
```

Literal widening — bites everyone once

```
const g = "male" // type: "male" ✓ const can't change, TS keeps it exact
let g = "male" // type: string ✗ let could be reassigned, so TS widens
```

Fix a widened literal with `as const`.

3.4 Decoding TypeScript errors

Error messages are a stack of "because" clauses. **Read them bottom-up** — the last line is the real cause.

Type 'string' is not assignable to type 'Gender'. ← THE REAL PROBLEM types of 'gender' are incompatible
 ← which property return types incompatible ← so the returned object is wrong not assignable to (prev: FormValues) => FormValues ← so your updater fn is wrong not assignable to SetStateAction<FormValues> ←
 so setFormValues rejects it

Error A — Argument of type 'null' is not assignable to parameter of type 'FormValues | (() => FormValues)'

Cause: `useState<FormValues>(null)`. You promised the state is always a `FormValues`, then handed it `null`.

- `FormValues` — the object itself.
- `() => FormValues` — a **lazy initialiser**: React calls it once on first render instead of every render. Useful when building the default is expensive.

Fix A (usually right): give real empty values. **Fix B**: widen to `useState<FormValues | null>(null)` — correct for *fetch*ed data, wrong for a form you type into.

Error B — Type 'string' is not assignable to type 'Gender'

Cause: `gender: e.target.value`. The DOM types `value` as plain `string`; it has no idea you wrote `value="male"` in the JSX. `string` is *wider* than `Gender`.

Fix	Code	Trade-off
Literal (best)	<code>gender: "male"</code>	fully type-safe; a typo is a compile error
Assertion	<code>e.target.value as Gender</code>	compiles, but erased at runtime — silences the check rather than satisfying it
Type guard	<code>(v): v is Gender => ...</code>	real runtime validation; right for API/URL data

Know the difference — common interview probe

Type assertion (`as`) is compile-time only. It is *not* a conversion and *not* a check — if the value isn't really a `Gender`, nothing catches it.

Type guard returns `v is Gender` — a **type predicate**. Inside a passing `if`, TypeScript **narrows** the variable to that type, and the check actually runs.

3.5 Narrowing — how the Context guard works

```
const context = useContext(TaskContext); // TaskContextType | null
if (!context) {
  throw new Error("..."); // throw ENDS execution
}
return context; // → TaskContextType, no null
```

Because `throw` ends execution, TypeScript knows every line after the `if` runs only when `context` is truthy — so it removes `null` from the type. One guard, written once, benefits every consumer.

Note: when TS can't resolve a property access it reports the result as `any` to avoid cascading errors — that's why the hover said `const taskList: any`.

3.6 React event types

Handler	Type
<code>onChange</code> on input	<code>React.ChangeEvent<HTMLInputElement></code>
<code>onChange</code> on select	<code>React.ChangeEvent<HTMLSelectElement></code>
<code>onChange</code> on textarea	<code>React.ChangeEvent<HTMLTextAreaElement></code>
<code>onSubmit</code> on form	<code>React.SubmitEvent<HTMLFormElement></code>
<code>onClick</code>	<code>React.MouseEvent<HTMLButtonElement></code>

The generic tells TS what `e.target` is — which is what makes `.value` and `.checked` exist on it.

FormEvent is deprecated (React 19 / @types/react 19.2+)

The message is blunt: "FormEvent doesn't actually exist." There is no browser event called "form" — only `submit`, `change`, `input`, `reset`. It was a catch-all bucket, and its declaration was **empty** (`interface FormEvent<T> extends SyntheticEvent<T> {}`) — a name that sounded specific but added no information.

`SubmitEvent` is real: it declares `target: EventTarget & HTMLFormElement`, so `new FormData(e.target)` works with no cast.

Note: `ChangeEvent` is *not* deprecated. Most tutorials still say `FormEvent` because it was correct for a decade.

Shortcut Inline handlers need no annotation at all — `<form onSubmit={e => ...}>` infers `e` from the prop. That's **contextual typing**: the surrounding expectation shapes the inference. You only annotate when the function is defined separately.

3.7 interface vs type

	interface	type
Object shapes	yes	yes
Unions	no	yes — "male" "female"
Declaration merging	yes — reopens and extends	no
Typical use	object/props shapes	unions, primitives, tuples, mapped types

Practical rule (and what the project does): `interface` for object shapes like `FormValues`, `type` for unions like `Gender` and `Role`.

4 JavaScript Essentials That Bite

4.1 "Literal" — two meanings

- **Everyday English:** exact, word-for-word, not a metaphor.
- **In code:** a value written directly, rather than computed or fetched.

```
"male" 42 true { name: "Ali" } [1, 2, 3] // Literals
someVariable getName() a + b e.target.value // NOT Literals
```

Test: can you see the actual value right there in the code? Then it's a literal.

4.2 Object identity — the single most important JS idea in React

```
"male" === "male" // true - primitives compare by VALUE
5 === 5           // true
{} === {}         // false - objects compare by IDENTITY, always
```

Two objects that look identical are still two objects. And **every time an object literal line runs, a brand new object is created:**

```
function makeUser() { return { name: "Ali" } }
const a = makeUser(), b = makeUser()
a.name === b.name // true - same contents
a === b           // false - different objects
```

Functions are objects too — so a function defined inside a component is recreated every render. This single fact explains `useMemo`, `useCallback`, `React.memo`, and dependency arrays.

4.3 The double braces in JSX

```
<TaskContext value={{taskList, addTask}}>
//           ↑ ↑
//           | └─ object literal - build an object
//           └─ JSX: "JavaScript goes here"
```

Also: `{taskList, addTask}` is **shorthand property** syntax for `{taskList: taskList, addTask: addTask}`.

4.4 Spread is shallow — one spread per level

`{...prev, addons: {...}}` does not *edit* addons — it **throws the old one away** and puts a new object there. Anything you don't list is gone.

```
// X Lunch and coffee VANISH (become undefined)
{ ...prev, addons: { overtime: true } }

// ✓ copy the inner object first, then override one key
{ ...prev, addons: { ...prev.addons, overtime: true } }
```

Why not just `prev.addons.overtime = true`? Because `...prev` copies the *reference* to the inner object, not the object. Mutating it changes the old state too, React sees no change, and the screen doesn't update. **Rule: one spread per level of nesting.**

4.5 Arrow functions returning an object literal

```
setFormValues((prev) => { ...prev, name: v } ) // X syntax error
setFormValues((prev) => ({ ...prev, name: v } )) // ✓ parens required
```

When JavaScript sees `{` right after `=>`, it always reads it as a **function body**, never an object. Parentheses can only contain an expression, so `{` inside them is unambiguously an object literal.

- `=>` then `{` means **code**. `=>` then `({` means **object**.
- Same fix in `.map()`: `items.map(x => ({ ...x, done: true } ))`
- Alternative: an explicit body with `return { ... }.`

4.6 Useful object helpers

```
Object.entries(task.addons) // [["overtime", true], ["Lunch", false], ...]
  .filter([, isOn] => isOn) // [, isOn] skips element 0 – a hole in the pattern
  .map([addon] => addon) // ["overtime"]

Object.values(task.addons).some(Boolean) // is anything true?

type Addon = keyof FormValues["addons"] // "overtime" | "Lunch" | "coffee"
```

`keyof` + an **indexed access type** pulls key names straight out of your interface, so a union stays in sync automatically when the interface changes.

4.7 Does everything return an object?

Two answers, because there are two levels to the question.

The JavaScript truth: almost everything is an object

```
typeof [] // "object" ← arrays ARE objects
typeof (() => {}) // "function" – but functions are objects too
typeof {} // "object"
typeof <div /> // "object" ← JSX too
```

Only **primitives** are not objects: `string`, `number`, `boolean`, `null`, `undefined`, `symbol`, `bigint`. That's why the identity rule applies to arrays and functions just as much as to plain objects.

JSX returns objects — this is what "virtual DOM" means

`<div />` compiles to a function call returning a **plain JavaScript object**:

```
{ type: 'div', props: {...}, key: null, ... }
```

Your components return objects. React compares those objects to work out what to change in the DOM. The "virtual DOM" is nothing more than a tree of ordinary JS objects.

But hooks deliberately return different shapes

Hook	Returns	Shape
<code>useState</code>	<code>[value, setter]</code>	array (tuple)
<code>useReducer</code>	<code>[state, dispatch]</code>	array
<code>useContext</code>	whatever you passed as <code>value</code>	yours to choose
<code>useRef</code>	<code>{ current: ... }</code>	object
<code>useMemo</code>	whatever your function returned	anything
<code>useCallback</code>	the function you gave it	function

Why `useState` returns an array but `context` returns an object

Array → destructuring by position lets *you* name both parts freely:

```
const [name, setName] = useState('')
const [count, setCount] = useState(0)    // rename at will

// with an object you'd be stuck renaming every time:
const { value: name, setValue: setName } = useState('')    // painful
```

Object → when you're returning three, five or ten things, order stops being memorable and names carry the meaning:

```
const { taskList, addTask } = useTaskContext()    // self-documenting
```

Convention for your own custom hooks Return an **array** for exactly two values where the order is obvious (a value and its setter). Return an **object** for three or more, or whenever names carry meaning.

5 The Context API

5.1 The problem: prop drilling

```
page.tsx ← taskList lives here └─ <Dashboard /> doesn't use it... but must accept and forward it └─  
    <Sidebar /> doesn't use it... but must accept and forward it └─ <TaskCount /> finally uses it
```

Prop drilling = threading a value through components that have no interest in it. Context skips the middle.

[Interview answer](#) Context lets you put a value **"in the air" over a subtree** so any component inside can read it directly, at any depth, without intermediate components passing it along. Props are handing a package person to person; context is a radio broadcast.

5.2 The three pieces

Piece	Job
<code>createContext()</code>	creates the channel
<code><Context value={...}></code>	broadcasts — wrap the subtree
<code>useContext(Context)</code>	receives — read from any depth

5.3 The complete typed pattern

```
"use client"
import React, { createContext, useCallback, useContext, useMemo, useState } from 'react'

export type Role = "admin" | "user";

export interface FormValues { name: string; task: string; role: Role; /* ... */ }

// 1. what the channel carries
interface TaskContextType {
  taskList: FormValues[],
  addTask: (task: FormValues) => void
}

// 2. create the channel
const TaskContext = createContext<TaskContextType | null>(null);

// 3. the provider – broadcasts to everything inside
const TaskProvider = ({children}: {children: React.ReactNode}) => {
  const [taskList, setTaskList] = useState<FormValues[]>([]);

  const addTask = useCallback((task: FormValues) => {
    setTaskList(prev => [...prev, task]);
  }, []);

  const value = useMemo(() => ({taskList, addTask}), [taskList, addTask]);

  return <TaskContext value={value}>{children}</TaskContext>
}

// 4. custom hook – how components consume it
export const useTaskContext = () => {
  const context = useContext(TaskContext);
  if (!context) throw new Error("useTaskContext must be used inside a TaskProvider");
  return context;
}

export default TaskProvider
```

Wrap the tree in `layout.tsx`, then consume anywhere:

```
<body><TaskProvider>{children}</TaskProvider></body>

// any component, any depth – no props
const { taskList, addTask } = useTaskContext()
```

5.4 Five details that matter

Detail	Why
<code>children</code> prop	The provider doesn't know what's inside — <code>children</code> is a hole. That's how one provider serves a whole app.
<code>createContext<T null>(null)</code>	The default is used when a consumer has no provider above it . You have no sensible default, so <code>null</code> is honest — and the type must allow it.
The custom hook guard	Turns a vague <code>Cannot read properties of null</code> into a clear message — and narrows away the <code>null</code> so no consumer needs <code>?.</code>
<code><Context value={}></code>	React 19 syntax — the context <i>is</i> the provider. Older code writes <code><Context.Provider></code> ; still works, no longer needed.
<code>useMemo</code> the value	Otherwise a fresh object each render makes every consumer re-render. See Part 6.

5.5 App Router rules (Next.js)

From the official Next.js docs **React context is not supported in Server Components.**

- **"use client" is mandatory** in the context file — `createContext`, `useContext` and `useState` are client-only.
- **But `layout.tsx` stays a Server Component.** A Server Component can *render* a Client Component — importing `<TaskProvider>` and wrapping `{children}` is fine.
- **`children` passed through the provider stay Server Components.** They're rendered on the server and slotted in. Wrapping in a provider does *not* turn your app into client code — a very common misconception.

5.6 When NOT to use context

- Two levels of prop passing is not prop drilling — it's just props.
- Use it for genuinely shared values: theme, logged-in user, language, cart, or a list many components read.
- **Context is not a state manager.** It only *transports* a value; `useState` in the provider is what stores it. No reducers, no persistence, no devtools.

6 Re-renders & Optimisation

6.1 What a re-render actually is

React calls your component function again, top to bottom. Every line re-runs; every function and object inside is recreated. Only hooks survive, because they reach into React's storage.

The only three triggers

1. Its own state changed (you called a setter)
2. Its parent re-rendered
3. A context it consumes changed

Key insight "A new object was created" is **not** on that list. A new object is a *consequence* of rendering, never a *cause* — which is why the context-value pattern can't loop infinitely. Data flows one way: providers push down, consumers never push back up.

Where it *can* become infinite

```
const value = useTaskContext()

useEffect(() => {
  setSomething(value.taskList.length)
}, [value])           // ← new object every render

// render → new object → dep "changed" → effect → setState → render → ∞
```

That's the return path the simple flow lacks — and the real reason to memoise a context value.

6.2 Why re-renders are cheap

A re-render is not a DOM update React re-runs your function to get fresh JSX — a plain JS description of the UI, not real DOM. It compares that to the previous description and **only touches the DOM where something differs** (*reconciliation*, the "virtual DOM" diff). A wasted re-render costs a function call and some comparisons — microseconds, not a repaint.

6.3 useMemo vs useCallback

Remembers...	
<code>useMemo</code>	the result of calling a function
<code>useCallback</code>	the function itself

```
const a = useMemo(() => 5 + 5, []) // a === 10
const b = useCallback(() => 5 + 5, []) // b === () => 5 + 5
```

`useMemo` **calls** your function and stores the output. `useCallback` **doesn't call it at all** — it just hands the same function back. They're the same hook:

```
useCallback(fn, deps)  ≡  useMemo(() => fn, deps)
```

`useCallback` exists purely so you don't have to write `useMemo(() => myFn, [])` — an arrow returning an arrow.

How they actually work — the whole mechanism

React keeps a **storage slot** for each `useMemo` / `useCallback` call in your component. On every render it does exactly this:

```
1. Compare the new deps array to the stored one, item by item, using Object.is
2. All identical? → return the STORED value. Don't run the function at all.
3. Any different? → run the function, store the new result + new deps, return it.
```

That is the entire implementation. No magic — a saved value and an array comparison.

Two consequences worth internalising

- **The deps array is the cache key.** Too few deps → a stale value. Unstable deps → no caching at all.
- **It is not free.** React still stores and compares on every render. You're trading a little work for a lot of work — only a win if the "lot" is real.

useMemo has two distinct jobs

Job	Example	You care about
Skip expensive work	<code>useMemo(() => list.filter(...).sort(...), [list])</code>	CPU time
Keep identity stable	<code>useMemo(() => ({taskList, addTask}), [...])</code>	reference equality

useCallback has one job: stable function identity

Never for speed — creating a function is nearly free. It matters in exactly two places:

1. **The function is in a dependency array** — otherwise the effect re-runs every render (see 6.1).
2. **The function is a prop on a `React.memo` component** — otherwise the memo compares a fresh function every render and skips nothing.

React.memo + useCallback are a pair One without the other is usually pointless. You paid for the comparison and got zero benefit.

The decision test

```
Does anything COMPARE this value to its previous version?
├ passed to a React.memo child → yes, memoise
├ listed in a deps array → yes, memoise
├ returned from a custom hook → yes, memoise
└ just wired to onClick / onChange → NO. Skip it.
Memoising a FUNCTION? → useCallback
Memoising a VALUE? → useMemo
```

6.4 The seven scenarios, explained

Theory is easy to forget. These are the situations you will actually meet.

useMemo · Scenario 1 — expensive derived data

Setup. You add a search box to `TaskList`:

```
const [search, setSearch] = useState('')

const filtered = taskList.filter(t =>
  t.task.toLowerCase().includes(search.toLowerCase())
)
```

What goes wrong. That line runs on every render of `TaskList` — and `TaskList` re-renders for reasons unrelated to search. With 20 tasks: irrelevant. With 5,000 tasks plus a `.sort()`, you're doing thousands of string operations between keystrokes and the input starts lagging.

With useMemo:

```
const filtered = useMemo(
  () => taskList.filter(t => t.task.toLowerCase().includes(search.toLowerCase())),
  [taskList, search]
)
```

Why it's different from the rest: here you care about **CPU time**. Identity is a side benefit. This is the only scenario about speed — the other six are all about identity.

useMemo · Scenario 2 — a context provider's value

Setup. `<TaskContext value={{taskList, addTask}}>`

What goes wrong. The object literal is a brand new object every time the provider renders. React compares context values by identity:

```
TaskProvider re-renders (for any reason) ↓ {taskList, addTask} → new object ↓ React: "value changed" (it
didn't really – same contents) ↓ EVERY consumer re-renders, even ones reading only addTask
```

Building the object is instant — cost isn't the issue. The issue is consumers can't tell "new object" from "new data".

Why providers specifically: a provider sits above a large subtree, so one unnecessary identity change fans out to everything below it.

useMemo · Scenario 3 — a value inside a dependency array

Setup.

```
const query = { role, gender }

useEffect(() => { fetchTasks(query) }, [query])
```

What goes wrong. This is the dangerous one:

```
render → query is a new object ↓ useEffect compares [query] to last render's → "changed!" ↓ effect runs  
→ fetchTasks → setState ↓ re-render → query is a NEW object again ↓ effect runs again → ... forever
```

This is the most important scenario An **infinite loop** — continuous network requests, frozen tab. The others cost performance; this one **breaks the app**. Any object or array in a deps array must be memoised or hoisted out of the component. And the code looks perfectly reasonable — the bug is invisible unless you know objects compare by identity.

useMemo · Scenario 4 — a prop on a React.memo child

```
const Table = React.memo(({ data, config }) => { /* expensive render */ })  
  
<Table data={tasks} config={{ pageSize: 10, sortBy }} /> // X
```

What goes wrong. `React.memo` skips re-rendering when props are unchanged — compared shallowly, by identity. `config` is a fresh object every render → "props changed" → `Table` re-renders every time. **You added `React.memo`, paid for the comparison, and skipped nothing.** Strictly worse than not using it.

```
const config = useMemo(() => ({ pageSize: 10, sortBy }), [sortBy]) // ✓
```

useCallback · Scenario 1 — a handler passed to a memoised child

```
const TaskCard = React.memo(({ task, onDelete }) => { ... })  
  
// X new function every render → all 50 cards re-render  
{tasks.map((t, i) => <TaskCard key={i} task={t} onDelete={() => remove(i)} />)}  
  
// ✓ stable → only the changed card re-renders  
const handleDelete = useCallback((i: number) => {  
  setTaskList(prev => prev.filter((_, idx) => idx !== i))  
}, [])  
{tasks.map((t, i) => <TaskCard key={i} task={t} onDelete={handleDelete} index={i} />)}
```

Two things happened there. **(1)** `[]` is safe because the function reads nothing from this render — `prev =>` earns the empty array. **(2)** The index moved to a prop instead of being baked into a closure; otherwise you'd need a different function per card and be back where you started.

useCallback · Scenario 2 — a function inside a dependency array

```
const loadTasks = useCallback(async () => {  
  const res = await fetch('/api/tasks')  
  setTaskList(await res.json())  
}, [])  
  
useEffect(() => { loadTasks() }, [loadTasks])
```

Without the `useCallback`, same infinite loop as `useMemo` Scenario 3. **You will definitely hit this** — it's the standard data-fetching shape, waiting for the first `useEffect` that calls a function defined in the component.

`useCallback` · Scenario 3 — a function returned from a custom hook

```
export const useTasks = () => {
  const [tasks, setTasks] = useState<FormValues[]>([])
  const addTask = useCallback((t: FormValues) => setTasks(prev => [...prev, t]), [])
  return { tasks, addTask }
}
```

You don't control how consumers use `addTask`. If someone writes `useEffect(..., [addTask])`, *their* effect loops — but the cause is in *your* file.

The principle Anything you **export** — from a custom hook, a context, a shared component — should have stable identity, because you can't see how it will be consumed. Inside one component you can see every usage and decide. Across a boundary you can't. **Stability is part of a public API.**

The unifying idea

Read all seven together and there are really only **two** reasons:

Reason	Which scenarios
The work is expensive	<code>useMemo</code> Scenario 1 only. You're saving CPU.
Something compares the identity	All six others — <code>React.memo</code> comparing props, <code>useEffect</code> comparing deps, React comparing context values. The work was never the problem; the fresh object was.

6.5 "But the dependency is in the array anyway..."

A natural objection to Scenario 1:

If search is in the deps, it recomputes on every keystroke anyway. What's the advantage?

It protects against **other** renders — not against search changing.

Recomputing when `search` changes isn't a failure — it's the point. If the search text changed, the results *must* change. `useMemo` makes sure that's the **only** time it happens.

The real question is: **what else was re-rendering this component?** Suppose `TaskList` also has an expand-card state and a sort toggle. With 5,000 tasks:

Action	Without useMemo	With useMemo
Type a letter in search	filter runs	filter runs — correct , results changed
Click a card to expand	filter runs ✗	skipped ✓
Toggle sort direction	filter runs ✗	skipped ✓
Hover state, modal open/close	filter runs ✗	skipped ✓
Someone submits the form	filter runs	filter runs — correct, <code>taskList</code> changed
Parent re-renders	filter runs ✗	skipped ✓

Every ✗ is 5,000 string operations producing exactly the array you already had.

The mental model **Without useMemo**: recompute on EVERY render. **With useMemo**: recompute only when the INPUTS change. The deps array is you declaring "this depends on `taskList` and `search`, nothing else" — every other reason to re-render is filtered out.

When it genuinely buys nothing

If the *only* thing that ever re-renders the component is the search input, the memo never skips anything and you've paid for storage plus a comparison. The real test isn't "is this expensive?" but:

VALUE = cost of the computation × renders that DON'T change its inputs
If either factor is near zero → skip it.

If typing itself is the slow part

`useMemo` can't help — `search` is a genuine dependency. That's a different problem with a different tool:

```
const deferredSearch = useDeferredValue(search)

const filtered = useMemo(
  () => taskList.filter(t => t.task.includes(deferredSearch)),
  [taskList, deferredSearch]
)
```

`useDeferredValue` lets the input update immediately at full speed while the expensive filtering lags a render or two behind. The field stays responsive; results catch up. (Debouncing is the older manual version.)

Remember the distinction **useMemo skips unnecessary work**. **useDeferredValue deprioritises necessary work**. Different problems, different tools.

6.6 Dependency arrays and stale closures

Deps	Meaning
<code>[]</code>	compute once, reuse forever
<code>[a, b]</code>	recompute when <code>a</code> or <code>b</code> changes (compared with <code>Object.is</code>)
<i>omitted</i>	no caching at all — recomputes every render

Stale closure — the real danger

```
const [count, setCount] = useState(0)
const logCount = useCallback(() => { console.log(count) }, []) // X
// click ten times → still logs 0
```

The function was created on render #1 and permanently captured that render's `count`. Empty deps said "never rebuild," so it's frozen in the past.

Rule: every render-scoped variable the function reads must be in the deps.

Why the project's `addTask` can safely use `[]`

```
const addTask = useCallback((task: FormValues) => {
  setTaskList(prev => [...prev, task]);
}, []);
```

Used	Render-scoped?
<code>setTaskList</code>	No — React guarantees state setters are stable forever
<code>prev</code>	No — React supplies it at call time
<code>task</code>	No — it's a parameter from the caller

The payoff of `prev =>` Writing `setTaskList([...taskList, task])` instead would capture `taskList` from the current render, making `[]` a stale-closure bug and forcing `[taskList]` — which defeats the memo. **The updater form is what earns the empty array.**

6.7 When NOT to memoise

Situation	Why skip it
<code>onClick={() => setOpen(true)}</code> inline	Nothing compares it. DOM handlers don't care about identity.
<code>handleSubmit</code> on your form	Same — React just wires up the latest one.
<code>const total = a + b</code>	Cheaper to compute than to memoise.
Memoising a primitive	<code>useMemo(() => 5, [])</code> — primitives compare by value; identity was never a problem.
Child is not wrapped in <code>React.memo</code>	It re-renders regardless. <code>useCallback</code> alone achieves nothing.
Deps change every render anyway	Worse than not memoising — you pay storage and comparison cost for zero benefit.

- Both hooks cost code, indentation, and a deps array you must keep correct.
- **Wrong deps cause stale bugs, which are far worse than an extra render.**
- React still stores and compares deps on every render — memoising isn't free.

Default to neither. Add them for a specific reason: a `React.memo` boundary, a dependency array, or a measured slow computation.

6.8 The 10-second checklist

Before writing either hook, ask in order:

#	Question	If...
1	Does anything compare this value's identity? (<code>React.memo</code> child, deps array, custom-hook return)	No to all → stop , use neither
2	Is the computation genuinely expensive? (loops over thousands, sorting, parsing)	No, and #1 was no → stop
3	Function or value?	Function → <code>useCallback</code> . Value → <code>useMemo</code>
4	Are my deps themselves stable?	If a dep is recreated every render, memoise <i>that</i> first or the whole thing is pointless
5	Does the function read anything from this render?	If yes it must be in the deps — or rewrite with <code>prev =></code> so it doesn't

Applied to the form-practice project Nothing currently qualifies: no `useEffect`, no `React.memo`, lists of a handful of items. The `TaskContext` memoisation is correct and good practice for a provider, but saves nothing measurable today. These checks start mattering the moment you add a search filter (→ `useMemo`), a delete button with memoised cards (→ `useCallback`), or data fetching (→ both).

React Compiler — where this is heading React 19 + Next.js 16 ship support for the **React Compiler**, which analyses components and inserts memoisation automatically, largely removing hand-written

useMemo/useCallback. It is **opt-in**: install babel-plugin-react-compiler and set reactCompiler: true in next.config.ts. This is why "just memo everything" was never good advice.

7 Next.js App Router Notes

7.1 Server vs Client Components

	Server Component (default)	Client Component
Marked by	nothing — it's the default	<code>"use client"</code> at the top of the file
Can use hooks?	No	Yes
Can use context?	No	Yes
Event handlers?	No	Yes
Ships JS to browser	No	Yes

`"use client"` goes **at the very top of the file, before any imports**. It marks an *entry point* — you don't need it in every file, only where the client boundary begins.

7.2 The boundary is one-directional

- A Server Component **can** import and render a Client Component.
- A Client Component **cannot** import a Server Component.
- Props crossing the boundary must be **serialisable** — no functions.

7.3 The payoff of splitting components

Before the split, `page.tsx` held all the state and needed `"use client"` — so the whole page shipped as client JS. After extracting `<Form />` and `<TaskList />`:

```
// app/page.tsx — no "use client", no hooks → a SERVER COMPONENT again
import Form from '../_components/Form'
import TaskList from '../_components/TaskList'

const Home = () => (
  <main>
    <header>...</header>
    <Form />
    <TaskList />
  </main>
)
```

- The layout, gradient, and header now render on the server and ship **zero JavaScript**.
- Only `Form` and `TaskList` become client bundles.
- **No props are passed** — they communicate entirely through context, and the component between them doesn't know they share anything.

The App Router pattern Keep Client Components as **small leaves**, not the whole tree. Push "use client" as far down as possible.

7.4 Private folders

A folder prefixed with `_` (e.g. `app/_components/`) is a **private folder** — excluded from routing, so it never becomes a URL segment. Without the underscore, App Router would treat the directory as a route.

7.5 Server Actions (the alternative to controlled forms)

React extends `<form>` so its `action` can be a Server Function, which receives a `FormData` object automatically:

```
<form action={createInvoice}>...</form>

async function createInvoice(formData: FormData) {
  'use server'
  const name = formData.get('name') // reads the `name` ATTRIBUTE
}
```

This is the uncontrolled route — `name` and `value` attributes become essential, and checkboxes need explicit `value`s (otherwise you get `"on"`). Always verify auth *inside* a Server Action.

8 Rapid-Fire Interview Q&A

What is a hook and why the name?

A function that lets a plain function component "hook into" React's internal features — state, lifecycle, context. Before hooks (React 16.8), those required class components.

Why can't hooks go inside an if?

React tracks state by **call order**, not by name. A conditional hook shifts every later hook's position, so state gets handed to the wrong variable.

Why does useState return an array?

So you can name both parts freely when destructuring — `[count, setCount]`, `[name, setName]`. An object would force fixed key names.

Why setState(prev => ...) instead of setState(value + 1)?

The plain variable is a **snapshot** from that render and doesn't update mid-handler, so two updates in a row both read the stale value. The updater receives the genuine latest value from React.

What is a controlled component?

An input whose value lives in React state rather than the DOM. Requires `value`/checked plus `onChange`. State paints the input; the input requests changes to state.

Why does an input freeze if you pass value without onChange?

React forces the input's displayed value back to state on every render. With no `onChange`, state never changes, so the typed character is overwritten instantly.

Difference between `e.target.value` and `e.target.checked`?

`value` is the fixed string written in the markup and never changes for a checkbox. `checked` is the boolean that actually toggles. Text inputs use `value`; checkboxes and radios use `checked`.

Why does the page reload when you submit a form?

Default HTML behaviour — with no `action`/`method`, the browser serialises the fields into a query string and navigates to the current URL. That full page load wipes React state. Fix with `e.preventDefault()`.

Why did some fields not appear in the submitted URL?

Inputs without a `name` attribute are not submitted at all. `id` links a label; `name` is the submission key. Unticked checkboxes are also absent entirely — they never send `false`.

What's the difference between shallow and deep copy in state updates?

Spread is **shallow** — it copies references to nested objects. To update a nested field you must spread each level: `{...prev, addons: {...prev.addons, overtime: true}}`.

Why must an arrow function returning an object be wrapped in parentheses?

JavaScript reads `{` after `=>` as a function body, not an object literal. Parens force expression context.

What is Context and when should you use it?

A way to broadcast a value to a whole subtree without prop drilling. Use for genuinely shared state — theme, user, language. Not for two levels of props, and it's a transport mechanism, not a state manager.

Why create `Context<T | null>(null)` and a custom hook?

The default is what consumers get with no provider above them, and there's no sensible default — so `null` is honest. The custom hook throws a clear error and **narrows the type**, so no consumer needs null checks.

Why does a context value need `useMemo`?

The object literal in `value={{a, b}}` is recreated every render. React compares context values by identity (`Object.is`), so every consumer re-renders — and any `useEffect` depending on it can loop infinitely.

Difference between `useMemo` and `useCallback`?

`useMemo` caches the **result** of a function; `useCallback` caches the **function itself**. `useCallback(fn, deps)` is exactly `useMemo(() => fn, deps)`.

When should you NOT use them?

When nothing compares the identity — no `React.memo` child, no deps array, no expensive computation. They cost readability and a deps array that must stay correct; wrong deps cause stale bugs worse than an extra render.

How does `useMemo` work internally?

React stores the value and the deps array in a slot. Each render it compares the new deps to the stored ones with `Object.is`. All the same → return the stored value without running the function. Any different → run it, store the new result and deps.

If the search text is in the deps, doesn't it recompute on every keystroke anyway?

Yes — and that's correct, the results genuinely changed. `useMemo` protects against **other** renders: expanding a card, toggling a sort, a parent re-rendering. It makes the inputs the *only* thing that triggers recomputation.

Why does useState return an array but Context return an object?

Array destructuring is positional, so you can name both parts freely (`[count, setCount]`). Objects are self-documenting, which matters once you're returning three or more things. Convention for custom hooks: array for two values, object for more.

Does everything in React return an object?

Nearly everything non-primitive is an object in JavaScript — arrays and functions included. JSX compiles to plain objects like `{type, props, key}`; that tree of objects is what "virtual DOM" means. But hooks deliberately return different shapes: arrays from `useState/useReducer`, a function from `useCallback`, whatever you choose from `Context`.

When is memoising worse than not memoising?

When the dependencies are themselves recreated every render. You pay the storage and comparison cost and the cache never hits. Same for wrapping a child in `React.memo` while passing it fresh object or function props.

Difference between useMemo and useDeferredValue?

`useMemo` **skips unnecessary work** — recompute only when inputs change. `useDeferredValue` **deprioritises necessary work** — let the input update instantly while expensive rendering lags a render behind.

What is a stale closure?

A memoised function that captured variables from an old render and never got rebuilt, so it keeps using outdated values. Caused by omitting dependencies.

What triggers a re-render?

Exactly three things: own state changed, parent re-rendered, or a consumed context changed. Creating a new object is a *consequence* of rendering, never a cause.

Is a re-render expensive?

Usually not. It re-runs the function and diffs the resulting JSX (reconciliation); only real differences touch the DOM. It's not a repaint.

Difference between a type assertion and a type guard?

`as` is compile-time only and erased at runtime — it silences the check. A type guard (`v is Gender`) performs a real runtime check and narrows the type inside the branch.

Why use interface for FormValues but type for Gender?

`interface` describes object shapes and supports declaration merging; `type` can express unions, which `interfaces` cannot.

Why can't Server Components use Context or useState?

They run only on the server and never hydrate, so there's no client-side instance to hold state or subscribe to updates. Mark the file "use client" to opt into the client.

9 Quick Reference

Binding cheat sheet

Input	Controlled binding	Handler reads
text	<code>value={state.name}</code>	<code>e.target.value</code>
select	<code>value={state.role}</code>	<code>e.target.value</code>
checkbox	<code>checked={state.addons.overtime}</code>	<code>e.target.checked</code>
radio	<code>checked={state.gender === "male"}</code>	<code>e.target.checked</code>

Hook selection

Need	Hook
value that survives renders and updates the UI	<code>useState</code>
value that survives but must NOT re-render	<code>useRef</code>
run code after render (fetch, timer, subscription)	<code>useEffect</code>
read a value broadcast from an ancestor	<code>useContext</code>
cache an expensive computation	<code>useMemo</code>
keep an object's identity stable	<code>useMemo</code>
keep a function's identity stable	<code>useCallback</code>
keep the UI responsive during expensive rendering	<code>useDeferredValue</code>
reuse stateful logic across components	a custom hook

Memoisation decision table

You have...	Reach for	Because
A filter/sort over a large list	<code>useMemo</code>	CPU cost
A context provider's <code>value</code> object	<code>useMemo</code>	identity fans out to the subtree
An object/array in a deps array	<code>useMemo</code>	otherwise the effect loops infinitely
An object prop on a <code>React.memo</code> child	<code>useMemo</code>	otherwise memo never skips
A handler prop on a <code>React.memo</code> child	<code>useCallback</code>	otherwise memo never skips
A function in a deps array	<code>useCallback</code>	otherwise the effect loops infinitely
A function exported from a custom hook	<code>useCallback</code>	stability is part of a public API
An inline <code>onClick</code> / <code>onChange</code>	neither	nothing compares its identity
Slow typing in a search box	<code>useDeferredValue</code>	the work is necessary, just low priority

Empty-value defaults

Type	Default	Never
text / textarea	<code>''</code>	<code>undefined</code> , <code>null</code>
checkbox	<code>false</code>	<code>undefined</code>
radio group	one member of the union	<code>''</code> (not in the union)
select	one member of the union	<code>''</code> unless added to the union
list	<code>useState<T[]>([])</code>	<code>useState([])</code> → <code>never[]</code>

Common errors → causes

Message	Cause
'null' is not assignable to 'FormValues (() => FormValues)'	<code>useState<FormValues>(null)</code> — widen the type or give real defaults
Type 'string' is not assignable to type 'Gender'	<code>e.target.value</code> is <code>string</code> ; use a literal, <code>as</code> , or a type guard
Property 'x' does not exist on type 'T null'	missing null guard — add <code>if (!ctx) throw</code> to narrow
'FormEvent' is deprecated	use <code>SubmitEvent<HTMLFormElement></code>
changing an uncontrolled input to be controlled	initial value was <code>undefined</code> ; use <code>''</code> / <code>false</code>
provided a <code>checked</code> prop without <code>onChange</code>	half-controlled input — add the handler
Radio won't reset after submit	missing <code>checked=</code> — data flows out but not back in

The mental models worth memorising

Concept	One-liner
Hook	the attachment point between your function and React's machinery
Controlled input	the input is a display screen; state is the truth
Spread	one spread per level of nesting
<code>prev =></code>	React hands you the real latest value, not a snapshot
Object identity	<code>{ } === { }</code> is <code>false</code> — always
Context	a radio broadcast, not a package passed hand to hand
Re-render	the function runs again; the DOM only changes where it differs
Memoisation	only worth it when something compares the identity